LAB 1 REPORT

Lab 1: DevOps Foundations & Continuous Integration | CO1

Exp 1 (5 marks): Install and Configure Git and Jenkins

Aim: Install Git on Ubuntu 24.04, verify, set global config.

```
$ sudo apt update
$ sudo apt install git -y
$ git --version  -> git version 2.43.0
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --list
$ sudo apt install openjdk-17-jdk -y
$ sudo apt install jenkins -y
$ sudo systemctl start jenkins
$ sudo systemctl status jenkins
```

Browser: http://localhost:8080 -> Unlock Jenkins

Verification: git --version shows version, Jenkins status active.

Observation: Git installed, Jenkins dashboard accessible.

Screenshot: Terminal git --version + Jenkins Unlock page

Exp 2 (5 marks): Version Control using Git

Aim: Init repo, stage, commit, branch, merge, push to GitHub.

```
$ mkdir sample-app && cd sample-app
$ git init
$ git add . && git commit -m "Initial commit"
$ git branch -M main
$ git checkout -b feature/add-hello
$ git add . && git commit -m "Add hello feature"
$ git checkout main
$ git merge feature/add-hello
$ git log --oneline --graph
$ git remote add origin https://github.com/USERNAME/sample-app.git
$ git push -u origin main
```

Observation: Branching keeps main stable, merge successful.

Screenshot: git log graph + GitHub repo page

LAB 1 REPORT

Lab 1: DevOps Foundations & Continuous Integration | CO1

Exp 3 (5 marks): Basic Jenkins CI/CD Pipeline

Aim: Pipeline pulls from GitHub and triggers automated build.

Steps:

1. Jenkins > New Item > Freestyle Project > sample-app
2. Source Code Management > Git > Repo URL paste

```
    Branch: */main
```

3. Build Triggers > GitHub hook trigger for GITScm polling
4. Build > Execute shell:

```
$ npm install
$ npm test
```

5. Save > Build Now -> Build #1 SUCCESS

Observation: Every push auto-triggers build. Manual build eliminated.

Screenshot: Jenkins Job Config + Build Console Success

Exp 4 (5 marks): Jenkinsfile - Declarative Pipeline

File: Jenkinsfile in repo root

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'npm install' } }
    stage('Test') { steps { sh 'npm test' } }
    stage('Deploy') { steps { sh './deploy.sh' } }
  }
  post { success { echo 'Pipeline succeeded!' } }
}
```

Jenkins: New Item > Pipeline > Pipeline script from SCM > Jenkinsfile

Result: Stage View shows Build/Test/Deploy green.

Observation: Pipeline-as-Code makes pipeline reproducible.

Screenshot: Jenkins Stage View (3 green stages)

LAB 1 REPORT

Lab 1: DevOps Foundations & Continuous Integration | CO1

Exp 5 (5 marks): Automate Application Deployment

Aim: Configure triggers, post-build actions, deployment.

Configuration:

- Build Triggers: Poll SCM -> H/5 * * * * (every 5 min)
- Post-build: Archive artifacts **/*.log, Email on failure
- Deploy Step:

```
$ scp -r . user@server:/var/www/html/sample-app
$ ssh user@server "pm2 restart sample-app"
```

deploy.sh:

```
#!/bin/bash
echo Deploying...
cp -r app.js /var/www/html/ || echo Deployed locally
echo Deployment done
```

Result: Push -> Auto Build -> Auto Deploy -> App live on localhost:3000

Observation: Zero manual deployment, rollback via previous build.

Screenshot: Deploy stage log + Live app URL

Exp 6 (5 marks): Record All Commands

This report itself is Exp 6 evidence.

Evidence: commands.txt, Jenkinsfile, deploy.sh, commit history,

```
pipeline config, screenshots list maintained in GitHub repo.
```

Practice: Meaningful commits, .gitignore for node_modules, README.md with setup instructions.

Exp 7 (10 marks): CI/CD Workflow Report

Architecture:

Developer -> Git Push -> GitHub -> Jenkins Webhook -> Build -> Test -> Deploy -> Production Server -> Feedback

Results: Total Builds 5, Success 5, Avg Time 42 sec, Manual Time Saved ~10 min per deploy.

Observations: Automation reduced human error, reproducible, branching protects main, easy rollback via Jenkins.

Conclusion: Complete DevOps foundation achieved.